

Inventory Management System — Full Explanation

Prepared for: Kavya S

Date: November 07, 2025

Inventory Management System — Full Explanation

Prepared for: Kavya S

Date: November 07, 2025

This document explains, in beginner-friendly language, every concept used in an Inventory Management System project built with Java, Spring Boot, Maven, JPA (Jakarta), MySQL, Postman, and JUnit. Each technology, term, and code file is defined, why it is used in the project, and how they fit together. The goal is to let you understand the project fully so you can explain it confidently in reports and viva.

1. What is an Inventory Management System?

An Inventory Management System is a software application that helps businesses track and manage items they keep in stock. It answers questions such as: What products do we have? How many units are left? Which items are low on stock? When should we reorder? In your project, the system provides APIs to add, update, delete, and view products. It can also detect low-stock items and (optionally) send an email alert to the admin.

2. Overview of Project Structure

The project follows a layered design, often described as Controller → Service → Repository. Each layer has a single responsibility:

- Controller (API layer): Receives HTTP requests from a client (Postman, frontend), validates input, and forwards requests to the service layer.
- Service (Business logic): Implements rules related to products (for example: if quantity < 4, flag low stock). It calls repository methods to read/write data.
- Repository (Data access): Uses JPA to read and write Java objects to the database tables.

This separation makes the code easier to maintain and test.

3. Java — The Programming Language

Definition:

Java is a widely used, object-oriented programming language. It runs on the Java Virtual Machine (JVM), which allows Java programs to run on many different systems without changing

the code.

Why Java for this project:

- Strong ecosystem for web development (Spring Boot).
- Mature libraries for database access, testing, and build automation.
- Good for teaching structured software engineering concepts.

How it is used:

All backend code (entities, services, controllers) is written in Java. A typical Java class in this project is a Product class that represents a product row in the database.

4. Spring Boot — Application Framework

Definition:

Spring Boot is a framework built on top of the Spring ecosystem that simplifies the creation of standalone, production-ready applications.

Key features Spring Boot provides:

- Embedded server (Tomcat) so you can run your application without installing a separate web server.
- Auto-configuration to reduce setup boilerplate (database connection, web endpoints, etc.).
- Starter dependencies to bring in common libraries easily.

Why used in this project:

- Quickly exposes REST APIs for CRUD operations on product data.
- Manages dependency injection (wiring of components) so you do not manually create instances.

Example in the project:

A class annotated with `@RestController` handles incoming HTTP requests. When you run the application, Spring Boot starts an HTTP server and listens on port 8080 by default.

5. REST API — Definitions and Examples

Definition:

REST (Representational State Transfer) is an architectural style for designing networked applications. REST APIs use HTTP methods (GET, POST, PUT, DELETE) to operate on resources (here, products).

Common API endpoints in the project:

- GET `/api/products` — Get a list of all products.
- POST `/api/products` — Create a new product (body: product data in JSON).
- GET `/api/products/{id}` — Get a single product by id.
- PUT `/api/products/{id}` — Update product details.
- DELETE `/api/products/{id}` — Remove a product.
- GET `/api/products/low-stock` — Get products with quantity below a chosen threshold.

How it works:

- A client sends an HTTP request to one of these endpoints.
- The Spring Boot controller method mapped to that endpoint is executed.
- The controller calls service methods which perform logic and use the repository to talk to the database.
- A response (usually JSON) is returned to the client.

6. Maven — Build and Dependency Management

Definition:

Maven is a build automation tool for Java projects. It uses a file named pom.xml to describe the project, its dependencies, and build steps.

What Maven does in this project:

- Downloads libraries (for example, Spring Boot starter libraries, JPA, MySQL connector).
- Compiles source code.
- Runs tests.
- Packages the application into a .jar file which can be run on any machine with Java installed.

Important pom.xml elements:

- — lists the libraries you need. Maven downloads them automatically.
- — tools that perform extra actions (for example, maven-surefire-plugin runs unit tests).

Typical Maven commands:

- mvn clean — delete previous build files.
- mvn install — compile, test, and put the package into the local Maven repository.
- mvn spring-boot:run — run a Spring Boot app directly from the source.

7. JPA / Jakarta Persistence (ORM)

Definition:

JPA (Jakarta Persistence API) is a Java specification for object-relational mapping (ORM). ORM lets you work with the database using Java objects rather than SQL queries.

How it helps:

- You write Java entity classes (like Product) and annotate them with @Entity. JPA maps those classes to database tables automatically.
- Common operations like save, find, delete can be performed through a Repository interface (for example, JpaRepository) without writing SQL.

Example entity (Product):

```
@Entity
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private int quantity;
```

```
private double price;
}
```

How persistence works in the project:

- When service calls `repository.save(product)`, JPA translates that into an INSERT or UPDATE SQL command to the database.

8. MySQL — Database

Definition:

MySQL is a popular open-source relational database management system (RDBMS).

Role in the project:

- Stores persistent product data (id, name, quantity, price, timestamps).
- Supports queries to retrieve low-stock items, search by name, or aggregate queries (like total inventory value).

Connection example in `application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost:3306/inventorydb
spring.datasource.username=root
spring.datasource.password=root
```

Note on developers' machines:

If you prefer not to install MySQL while testing, you can use H2 (an in-memory database) for quick tests. H2 keeps data only while the app is running.

9. Lombok — Reducing Boilerplate

Definition:

Lombok is a Java library that generates common code (getters, setters, constructors) at compile time using annotations.

Common Lombok annotations:

- `@Data` — generates getters, setters, `toString`, `equals`, and `hashCode`.
- `@NoArgsConstructor` / `@AllArgsConstructor` — generate constructors.

Why use it:

- Save time and avoid repetitive code.
- Keep the entity classes short and readable.

Important: IDE setup

- IDEs like IntelliJ or Eclipse require a Lombok plugin for proper code recognition while editing. However, code compiles fine with Lombok on the classpath.

10. Postman — API Testing Tool

Definition:

Postman is a tool to manually send HTTP requests to APIs and inspect responses.

How to use in this project:

- Use Postman to test endpoints before building or connecting a front end.
- Example: Send a POST request to `/api/products` with a JSON body to add a product, then send GET `/api/products` to verify it is saved.

Why Postman is useful:

- Quick verification of endpoints.
- You can save requests as a collection and replay them later (useful for demos).

11. JUnit — Testing Framework (Detailed)

Definition:

JUnit is the most common unit testing framework for Java. Unit tests verify small units of code (usually single methods) behave as expected.

Types of tests in the project:

- Unit tests: Test service methods in isolation using mocks for the repository.
- Integration tests: Run parts of the application together (for example, service + repository) to check whether they work as a set.

Sample unit test scenario:

1. `testAddProduct()` — create a product, call `productService.saveProduct(product)` and check that a product id is returned.
2. `testDeleteProduct()` — delete a product and assert that `repository.findById(id)` returns empty.
3. `testLowStockAlert()` — add a low stock product and verify the logic that finds low-stock items returns it.

Why tests matter:

- Detect bugs early.
- Make refactoring safer (you can change code confidently if tests pass).

12. Email Notification Feature — How It Works

Overview:

The project can be extended to send automatic email notifications to an admin when product quantity falls below a threshold.

Components involved:

- A service that checks quantities (could be triggered every time a product is updated or by a scheduled task).
- `JavaMailSender` provided by Spring Boot Starter Mail configured in `application.properties`.

Basic logic example:

```
if (product.getQuantity() < THRESHOLD) {  
    emailService.sendLowStockAlert(product);  
}
```

```
}
```

Security note:

When using Gmail, you should use an application password or a secure method to store credentials rather than plain text in `application.properties`.

13. `application.properties` — Configuration File

Purpose:

This file stores configuration keys for the Spring Boot application such as database URL, credentials, JPA options, and mail server settings.

Common keys used in the project:

- `spring.datasource.url` — JDBC URL to your MySQL database.
- `spring.datasource.username` — database username.
- `spring.datasource.password` — database password.
- `spring.jpa.hibernate.ddl-auto` — controls whether Hibernate should create/update tables automatically (none, validate, update, create, create-drop).

Important advice:

Never commit real credentials to a public GitHub repository. Use environment variables or a separate config that is not shared when deploying production apps.

14. Example Java Classes and What They Do

Product (Entity):

- Fields: `id`, `name`, `quantity`, `price`, maybe `createdAt/updatedAt`.
- Purpose: Model a product as a Java object and as a database row.

ProductRepository (Repository Interface):

- Extends `JpaRepository`
- Purpose: Provides methods like `save()`, `findAll()`, `findById()`, `deleteById()`, and custom queries such as `findByQuantityLessThan(int limit)`.

ProductService (Service Class):

- Purpose: Contains business rules and calls repository methods. Example methods: `addProduct()`, `getAllProducts()`, `deleteProduct()`, `getLowStockItems()`.

ProductController (Controller Class):

- Purpose: Exposes REST endpoints. It maps URLs and HTTP methods to `ProductService` methods and returns responses in JSON format.

15. Typical SQL Table Structure (Example)

Table: `products`

Columns:

- id (BIGINT, Primary Key, Auto Increment)
- name (VARCHAR)
- quantity (INT)
- price (DECIMAL)
- created_at (TIMESTAMP)
- updated_at (TIMESTAMP)

Sample SQL (created automatically by JPA when configured):

```
CREATE TABLE products (
id BIGINT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(255),
quantity INT,
price DECIMAL(10,2),
created_at TIMESTAMP,
updated_at TIMESTAMP
);
```

Note: You do not usually write this SQL manually when using JPA; JPA generates it from the entity classes.

16. Maven Lifecycle and Commands (Explained)

Lifecycle phases (the ones you commonly use):

- clean — removes previous build output.
- compile — compiles Java source code.
- test — runs unit tests.
- package — packages compiled code into a jar/war file.
- install — installs the package into the local Maven repository (useful if other projects depend on it).

Useful commands:

- mvn clean install — full build: clean, compile, test, package, and install.
- mvn spring-boot:run — run the Spring Boot application without building a jar explicitly.

17. How To Test APIs Using Postman (Step-by-step)

1. Start the Spring Boot application: mvn spring-boot:run.

2. Open Postman.

3. To add a product (POST):

- Set method to POST, URL to `http://localhost:8080/api/products`
- Under Body → raw → select JSON and paste:

```
{
"name": "Apple",
"quantity": 10,
"price": 50.0
}
```

- Click Send. You should receive a response containing the saved product and an id.

4. To get products (GET):

- Set method to GET, URL to `http://localhost:8080/api/products`
- Click Send. You should see a JSON array of products.

5. To delete a product (DELETE):

- Set method to DELETE, URL to `http://localhost:8080/api/products/{id}`
- Click Send and verify the product is removed.

18. Common Errors and How to Fix Them

1. Database connection errors:

- Symptom: Application fails to start with connection refused.
- Fix: Check MySQL is running and connection details in `application.properties` are correct.

2. Lombok-related IDE errors:

- Symptom: IDE shows missing getter/setter but code compiles.
- Fix: Install Lombok plugin in your IDE and enable annotation processing.

3. Maven dependency conflicts:

- Symptom: 'dependency must be unique' warning or classpath issues.
- Fix: Check `pom.xml` for duplicate dependencies and ensure versions are compatible.

4. Tests failing locally but not on CI:

- Fix: Ensure test data and database state are consistent. Use an in-memory database like H2 for unit/integration tests to isolate environment differences.

19. How to Explain This in Viva / Report (Short Script)

Use this script to explain the project verbally:

"Hello, my project is an Inventory Management System built with Java and Spring Boot. It follows a layered architecture: Controller, Service, and Repository. The application exposes REST APIs to perform CRUD operations on products. Data is stored in a MySQL database using JPA for ORM, which maps Java objects to database tables automatically. Maven is used for dependency and build management, and JUnit is used for automated testing. I used Postman to test APIs during development. The application can detect low-stock items and send email notifications to the admin."

Then be ready to explain one endpoint flow (for example, how a POST request to add a product flows through the layers) and show a sample test case.

20. Security and Best Practices (Brief)

- Do not store plain text passwords or secrets in source control. Use environment variables or external configuration.
- Validate user input in controllers to prevent invalid data from reaching the database.
- Use exception handling (`GlobalExceptionHandler`) to return friendly error messages and HTTP

status codes.

- Add logging for important actions (product added, deleted, low stock alert sent) for debugging and audit.
- Write unit and integration tests to ensure code quality.

Optional improvements:

- Add authentication (Spring Security) so only authorized users can change inventory.
- Add a simple frontend or connect to React/Angular to build a user interface.
- Add pagination and search for large lists of products.

21. Conclusion

This Inventory Management System project demonstrates a complete Java backend application using modern tools: Spring Boot (for APIs and app lifecycle), JPA (ORM), MySQL (data storage), Maven (build tool), JUnit (testing), Lombok (code brevity), and Postman (API testing).

Understanding each component's purpose and how they integrate will allow you to confidently explain the project in your report or viva. You can use the included example code structures, commands, and test cases while presenting or submitting the project.

Appendix: Useful Commands and Short Examples

Create database:

- `mysql -u root -p`
- `CREATE DATABASE inventorydb;`

Maven commands:

- `mvn clean install`
- `mvn spring-boot:run`

Sample curl commands:

- Add product:
`curl -X POST http://localhost:8080/api/products -H "Content-Type: application/json" -d '{"name":"Apple","quantity":10,"price":50.0}'`

- Get products:
`curl http://localhost:8080/api/products`

Sample Product.java (short):

```
@Entity
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private int quantity;
    private double price;
}
```

End of document.

